

ARCHITECTURE DES PROCESSEURS

M2 Secil : KINN9HU
Pascal SAINRAT

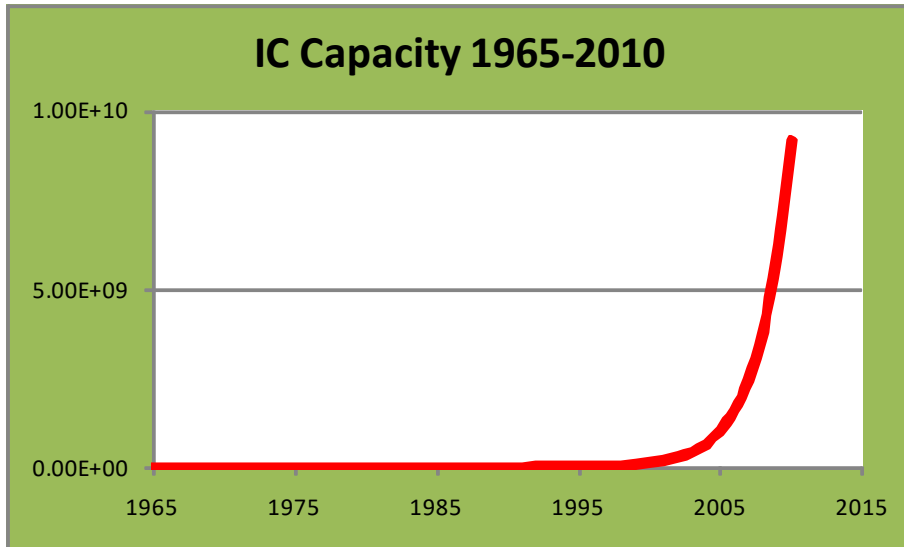


UNIVERSITÉ
TOULOUSE III
PAUL SABATIER



Faculté
Sciences
et **Ingénierie**

- Gérer la complexité des processeurs d'aujourd'hui repose sur des couches d'abstraction



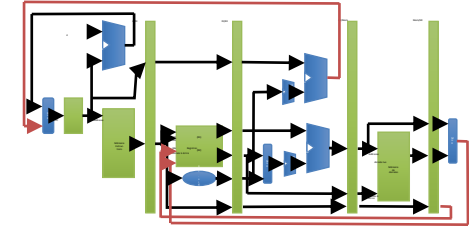
- Amélioration exponentielle démarrée il y a une cinquantaine d'années
 - Générateur de la performance, du coût, des fonctionnalités

Système et applications

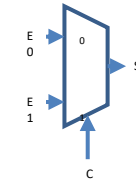
Architecture

```
LDR R3, [R6]
ADD R4, R3, R2
ADD R2, R2, #1
```

Microarchitecture



Fonctions logiques



Portes logiques



Transistors



Physique



Abstraction

Histoire des semiconducteurs

Date	Événement	Commentaires
1947	1 ^{er} transistor	Bell Labs
1958	1 ^{er} IC	Jack Kilby (Texas Instruments) Winner of 2000 Nobel prize
1971	1 ^{er} microprocesseur	Intel 4004
1974	Intel 4040	3 000 transistors
1978	Intel 8086	29 000 transistors
1989	Intel 80486	1 million de transistors
1995	Intel Pentium Pro	5,5 millions de transistors
2006	Intel Montecito	1,7 milliard de transistors
2022	Apple M2	20 milliards de transistors

Et Robert Noyce
(Fairchild) la même
année...

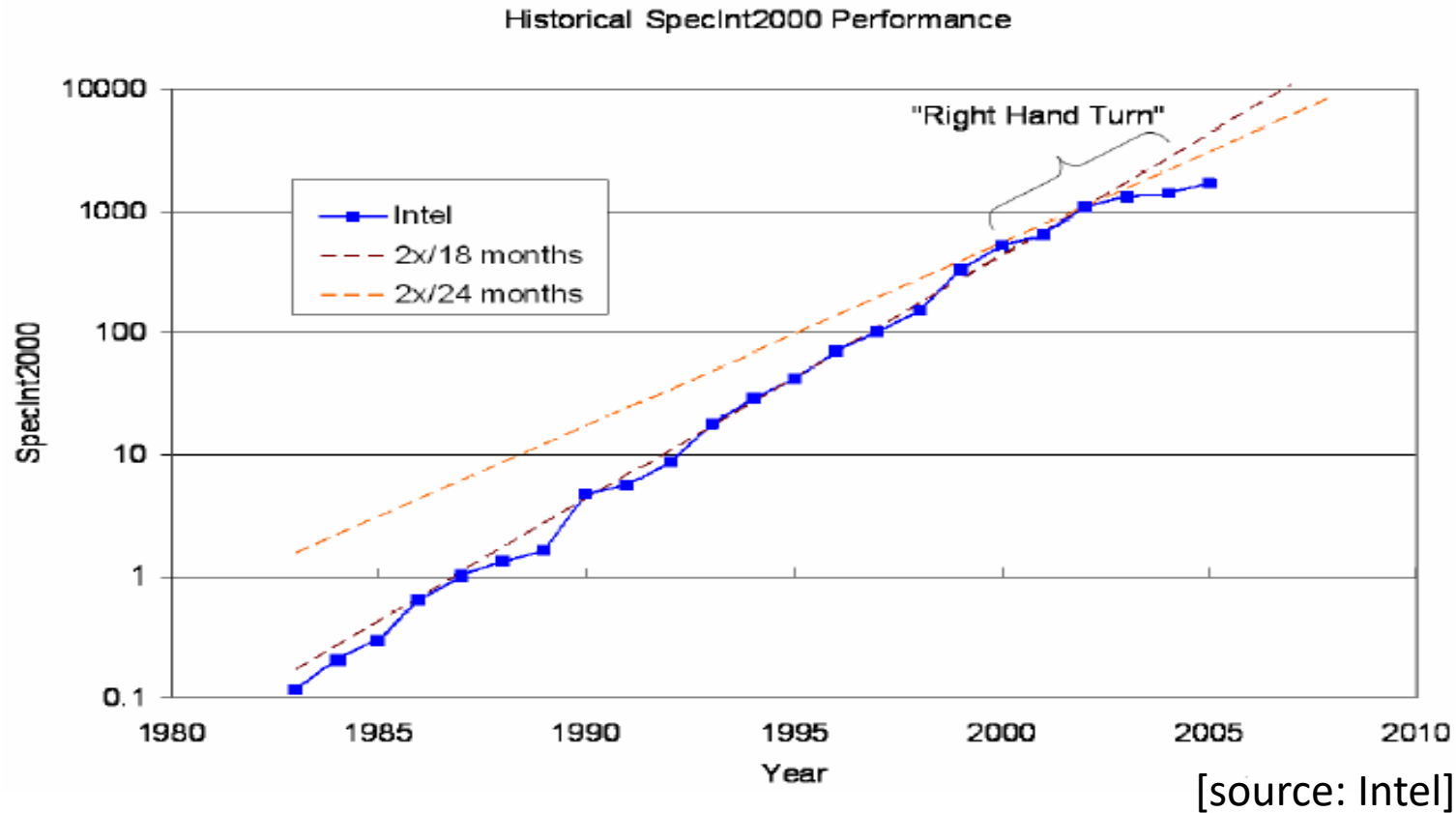
Intel fondée en 1968
par Robert Noyce,
Gordon Moore,
Andrew Grove

cœurs conçus par
Apple sous licence
ARM

Technologie et performance

- Loi de Moore
 - Le nombre de transistors double tous les 18 à 24 mois
- Les architectes les utilisent pour
 - améliorer les performances
 - réaliser des "économies" d'énergie
 - ajouter de nouvelles fonctionnalités
- La performance a doublé tous les 18 mois entre 1982 et 1996
 - Si on avait eu la même progression pour les voitures, elles rouleraient à 70 000 km/h ! et permettraient de parcourir plus de 3 000 kms avec un litre d'essence
 - En avion, on irait de Paris à New-York en moins d'une minute...
- Pas la même progression selon les technologies
 - Capacité DRAM : augmentation de 60%/an
 - Vitesse DRAM : seulement 10%/an => aujourd'hui très inférieure à la vitesse du processeur
- Points d'inflexion
 - Consommation, dissipation d'énergie, fiabilité, packaging

Evolution



- 2000 : fréquence stagne, "besoin" de plus de parallélisme, nouvelles applications
 - multi-cœurs, intégration GPU

Puissance dynamique

$$P_{dyn} \approx \sum_{i \in units} k_i C_i V^2 A_i f$$


- C : capacité du circuit
 - Longueur des fils, nombre et taille des transistors
- V : tension d'alimentation
- A : facteur d'activité (changements de valeurs)
- f : fréquence

Evolution vers le multicoeur

- Serveurs en 2001 (IBM power 4)
- PC de bureaux en 2005 (AMD Athlon X2)
- Ordinateurs portables en 2006 (intel core duo)
- Téléphones en 2009 (cortex A5)

	Single Core	Dual Core	Quad Core
Surface coeur	A	$\sim A/2$	$\sim A/4$
Puissance coeur	W	$\sim W/2$	$\sim W/4$
Puissance puce	W + O	W + O'	W + O''
Performance coeur	P	0.9P	0.8P
Performance puce	P	1.8P	3.2P

Loi d'Amdahl



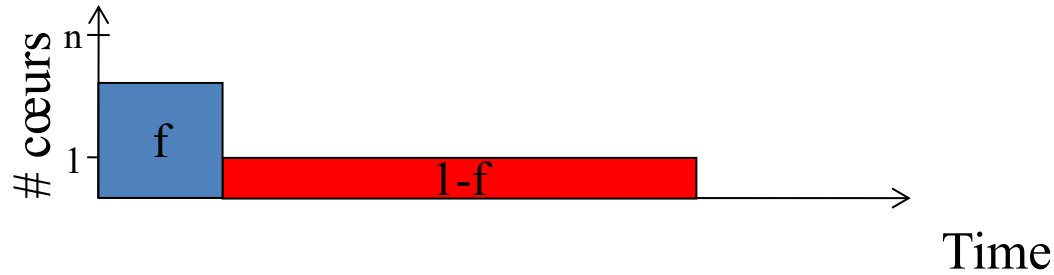
f – fraction qui peut être parallélisée

$1-f$ – fraction séquentielle

$$\text{Amélioration} = \frac{1}{(1-f) + \frac{f}{n}}$$

$$\lim_{n \rightarrow \infty} \frac{1}{1-f + \frac{f}{n}} = \frac{1}{1-f}$$

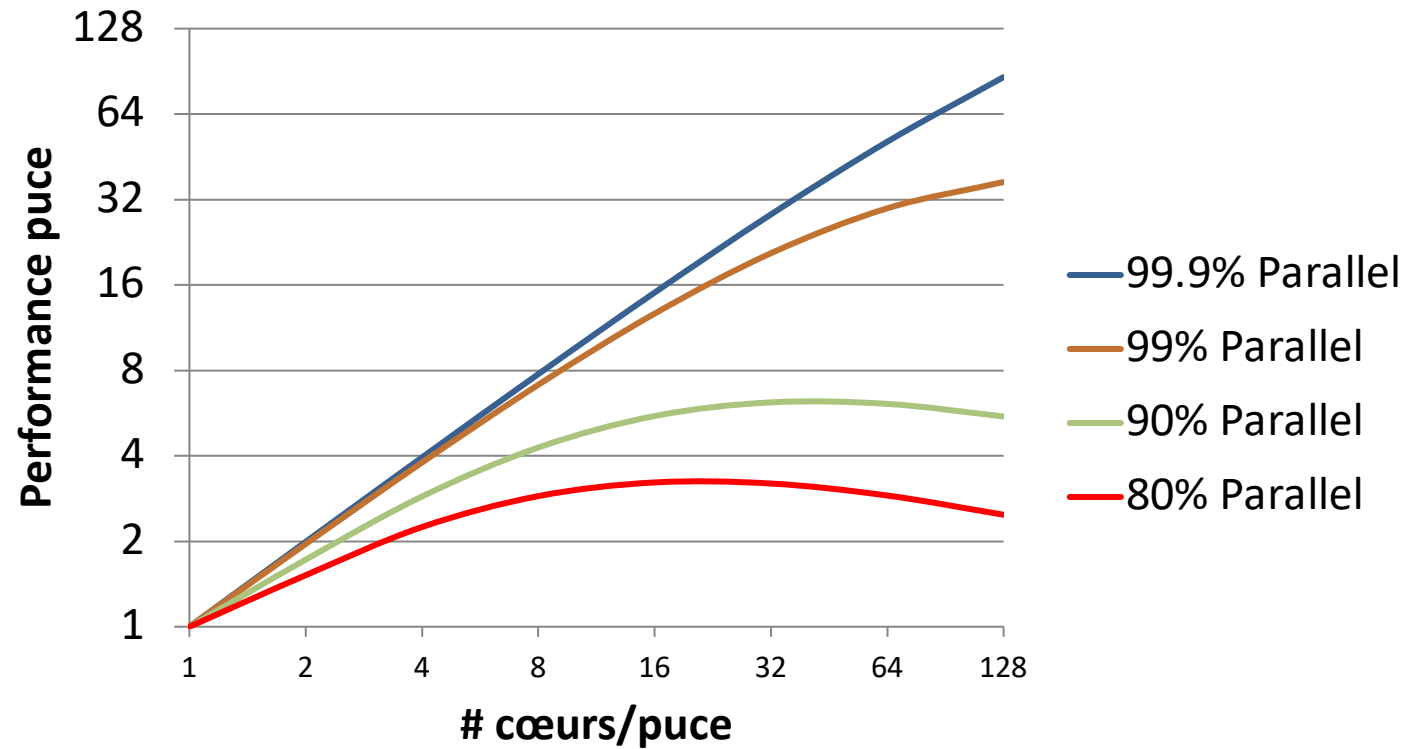
Budget de puissance fixe



- Loi d'Amdahl
 - Ignore le coût en puissance de n cœurs
- Loi d'Amdahl révisée
 - Plus de cœurs $\rightarrow$ chaque cœur est plus lent
 - Accélération parallèle $< n$
 - La portion série (1-f) prend plus de temps
 - En outre, surcharge d'interconnexion et de mise à l'échelle

$$speedup < \frac{1}{(1-f) + \frac{f}{n}}$$

Budget de puissance fixe



- Le budget de puissance fixe limite la fréquence de chaque cœur
- Le code séquentiel domine rapidement

Loi d'airain (monocoeur)

$$Performance\ du\ processeur = \frac{Instructions}{Programme} \times \frac{Cycles}{Instruction} \times \frac{Temps}{Cycle}$$

(Taille du code) (CPI) (Temps de cycle)

Compilateur μarchi techno

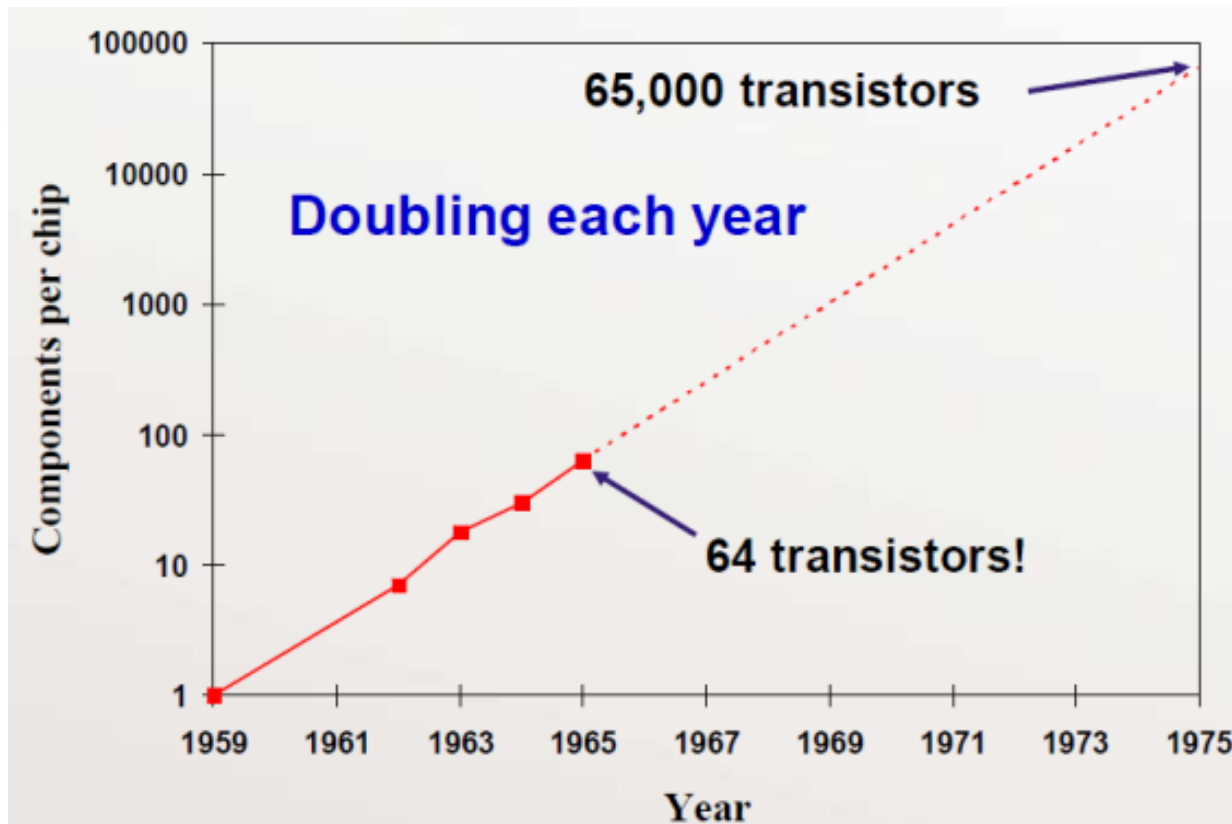
- Instructions/programme
 - Nombre d'instructions exécutées ← algorithme, compilateur, jeu d'instructions
- Cycles/instruction
 - jeu d'instructions, micro-architecture
 - Contraint par l'énergie par instruction
- Temps/cycle
 - technologie, micro-architecture, conception du circuit
 - Contraint par la limitation en puissance

Pré-requis

- Micro-Architecture et Conception des Microprocesseurs ou équivalent
 - Conception d'un processeur simple
 - Base jeu d'instruction RISC
 - Conception d'un chemin de données

Challenges technologiques

- La loi de Moore date de 1965.



Loi d'évolution de Dennard (1974)

- A chaque nouvelle génération technologique
 - Les dimensions d'un transistor diminuent de 30%
 - La surface diminue de 50%
 - La capacité C diminue de 30%
 - La tension doit être réduite de 30%
 - Le courant et le temps de transition diminuent de 30%
 - La fréquence augmente de 40%
 - La consommation d'énergie diminue de 50% (CV^2f)
- La surface et la consommation d'énergie sont divisées par 2 donc même énergie par unité de surface

Fin de la loi de Dennard...

- Avant, un monde parfait
 - La densité doublait tous les 2 ans (loi de Moore)
 - La vitesse de changement d'état s'améliorait de 30 à 40% à chaque génération
 - La tension d'alimentation et de seuil diminuaient
- A partir des années 2000
 - La taille et la densité continuent d'évoluer (aujourd'hui 5nm)
 - La vitesse de changement d'état s'améliore de 20% voire moins à chaque génération
 - La diminution de la tension d'alimentation est de moins en moins grande
 - Problèmes de fiabilité
- La dissipation de l'énergie pose de plus en plus de problèmes

En résumé

- La loi de Moore se vérifie encore : doublement du nombre de transistors tous les 2 ans
- La loi de Dennard ne marche plus : problèmes de dissipation de puissance
- La fiabilité des transistors empire : on pouvait la masquer avec de l'ECC par exemple mais cela devient de plus en plus difficile
- La communication entre puces (voire au sein de la puce) devient plus chère que le calcul lui-même
- Le coût de fabrication est de plus en plus difficile à amortir

Réduire la puissance dynamique

- Diminuer C
 - design plus simple, plus petit $\Rightarrow$ IPC réduit
- Diminuer A
 - design plus intelligent $\Rightarrow$ IPC réduit
- Diminuer la fréquence
 - Souvent en conjonction avec la baisse de la tension
- Diminuer la tension
 - Le plus efficace puisque V au carré
 - Peut-être statique
 - Ou dynamique (Intel speedstep par exemple)

$$P_{dyn} \approx kCV^2Af$$

Relation fréquence/tension

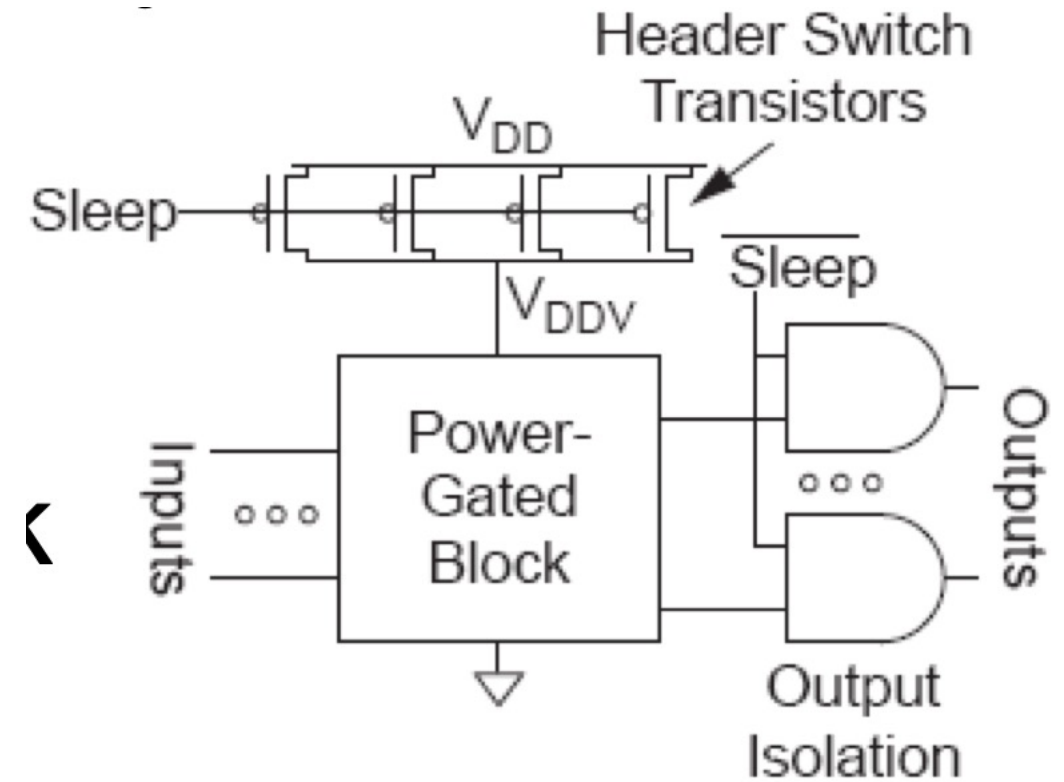
- Pour un même transistor, baisser la tension baisse la fréquence
- Et inversement : c'est le principe de l'overclocking
- Les puces sont triées pour déterminer à quelle tension nominale et donc à quelle fréquence elles peuvent fonctionner
 - approche semble très empirique mais gardée secrète
 - Impact sur la fiabilité
 - d'où l'overclocking possible (avec risque sur la durée de vie de la puce)
- Une variation de 1% de la performance entraine une variation de 3% de la puissance électrique dans le même sens
- Une technique pour économiser de l'énergie doit économiser plus de 3% de l'énergie pour une perte de 1% de performance sinon il est plus simple de baisser la tension.

Puissance statique

- Les transistors ne sont pas des interrupteurs parfaits, il y a toujours un courant qui passe
- Les solutions sont très technologiques (ex : haut diélectrique)
 - Sauf
 - à couper totalement des transistors
 - Faire des îlots de tension, parties non critiques plus lentes = nécessité de fournir plusieurs tensions
 - GALS : globalement asynchrone, localement synchrone
- Le rapport puissance statique/puissance électrique se rapproche de 1

Techniques architecturales (1)

- Clock gating
 - Couper l'horloge de certaines zones qui ne vont pas servir durant quelques temps
 - réduit la puissance dynamique
 - Largement utilisé aujourd'hui
- Power gating
 - Couper l'alimentation de blocs (par exemple, un cœur, une partie d'un cache, une unité fonctionnelle)
 - réduit la puissance statique
 - Utilisé aujourd'hui
- Dans les 2 cas, la gestion du mécanisme introduit un overhead



Techniques architecturales (2)

- Reconfiguration du cache
 - baisse de la puissance statique
 - Certaines applications peuvent se satisfaire d'un cache de taille moindre
 - Coupure de l'alimentation de certaines parties du cache
 - Conservation ou non de l'état
 - Complique un chemin critique (accès au cache L1)
 - Pas intéressant pour les caches de niveau supérieur car les transistors sont plus lents, la tension de seuil est plus haute, les transistors ont donc moins de courant de fuite

Techniques architecturales (3)

- Cœurs hétérogènes
 - Un cœur complexe pour les threads le nécessitant et de petits cœurs pour les autres
 - ex : ARM big.little
- Cœurs configurables ?
- Filter cache (économie d'énergie)
 - Un tout petit cache de niveau 1 pour éviter des accès à un cache plus gros quand ce n'est pas nécessaire
- Les idées fourmillent dans les conférences IEEE ou ACM
 - ISLPED, Micro, ISCA, HPCA Asplos, PACT